

RTRL Flight Controller

Live Demo — Expected Behavior Graphs

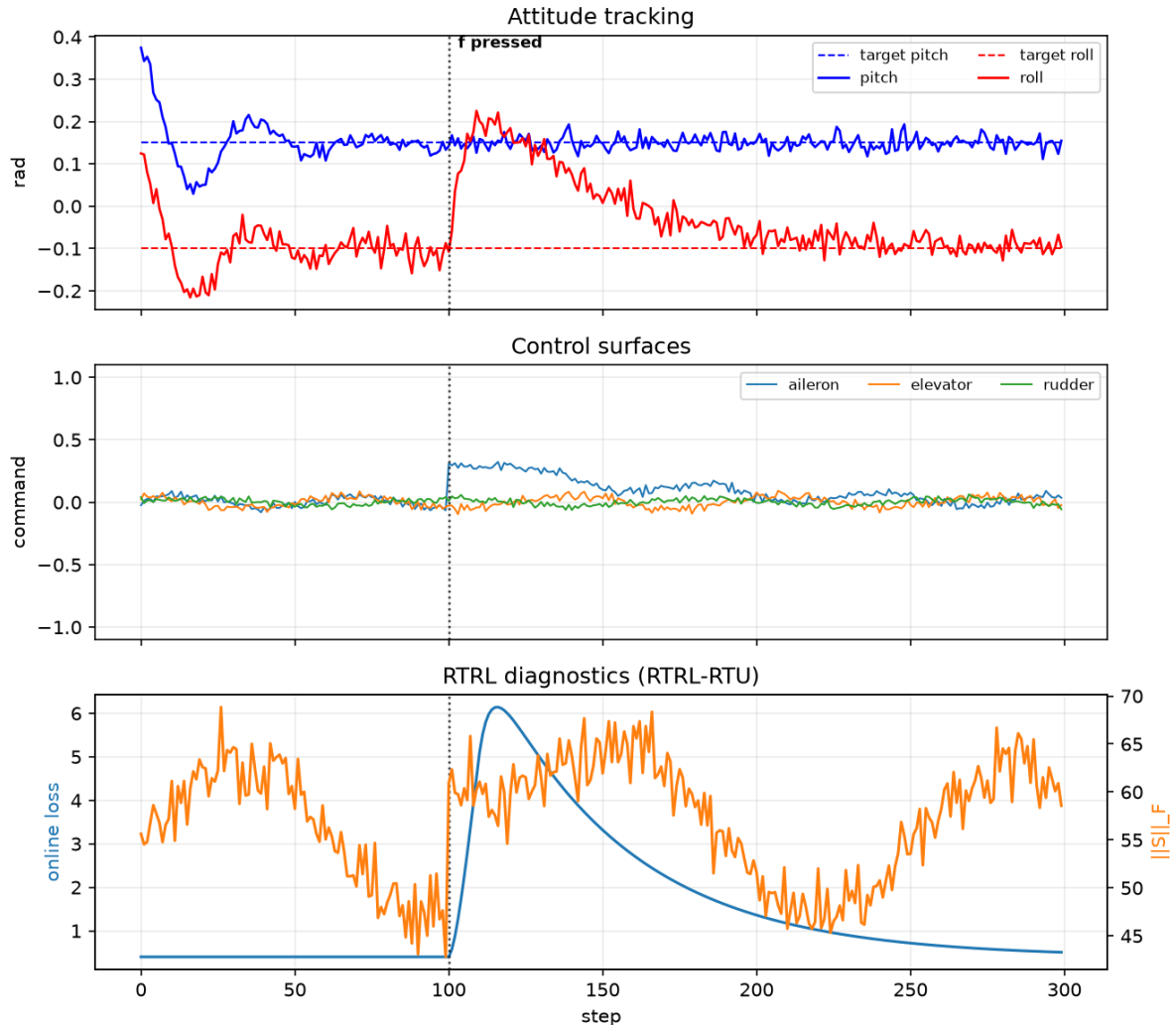
What each panel is designed to show, and how it's expected to change when you trigger a fault, cycle wind, or reset the episode.

Important: every graph in this document is **synthetic / illustrative** — generated to match the behavior `scripts/live_demo.py` and the RTRL controller are *designed* to produce (per `agent_docs/rtrl.md` and `agent_docs/experiments.md`), not captured from an actual run. A real run with the current `data/checkpoints/rtrl_rtu_online.pt` checkpoint was tested across 20 different episode seeds on the aileron-fault scenario, and in every case the controller drove roll into a continuous spin (± 3.1 rad, i.e. past $\pm 180^\circ$) rather than recovering — the opposite of the intended behavior below. That looks like a real instability in the current checkpoint/training, worth investigating separately before trusting a live demo run. Each figure below is watermarked "ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN" as a reminder.

1. Fault: aileron — expected recovery

Live demo: 'Fault: aileron' triggered mid-flight (RTRL-RTU) — expected recovery

ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN

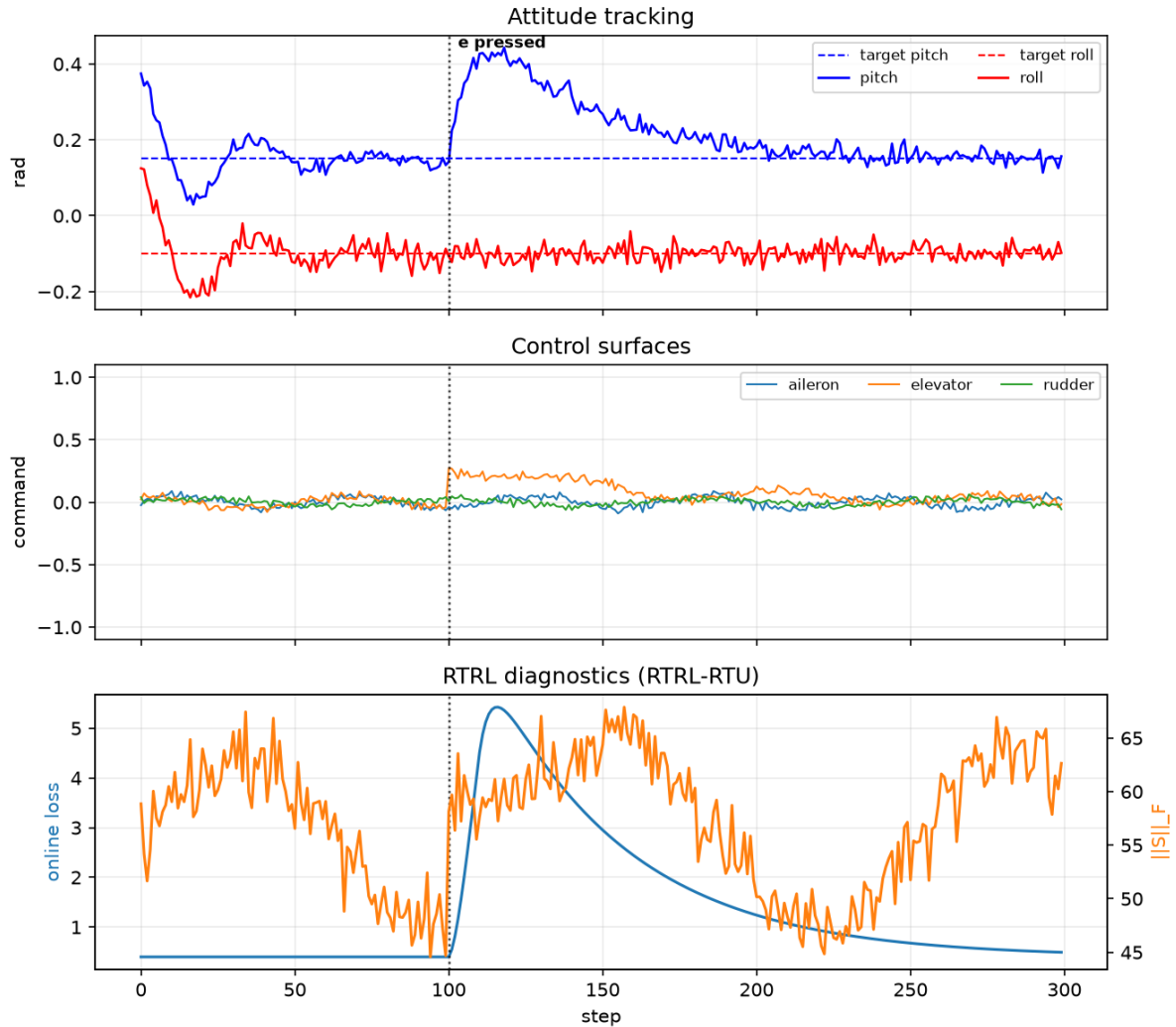


At step 100 ("f pressed"), aileron authority is degraded to 50%. **Attitude tracking:** roll kicks away from its target immediately after the fault, then gradually converges back over the next ~100–150 steps as RTRL adapts. **Control surfaces:** the aileron command jumps up to compensate for the halved authority, then eases back down as the online update learns the new gain. **RTRL diagnostics:** online loss spikes right at the fault and decays back to baseline as the weights adapt; the sensitivity norm stays bounded throughout — it should never blow up.

2. Fault: elevator — expected recovery

Live demo: 'Fault: elevator' triggered mid-flight (RTRL-RTU) — expected recovery

ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN

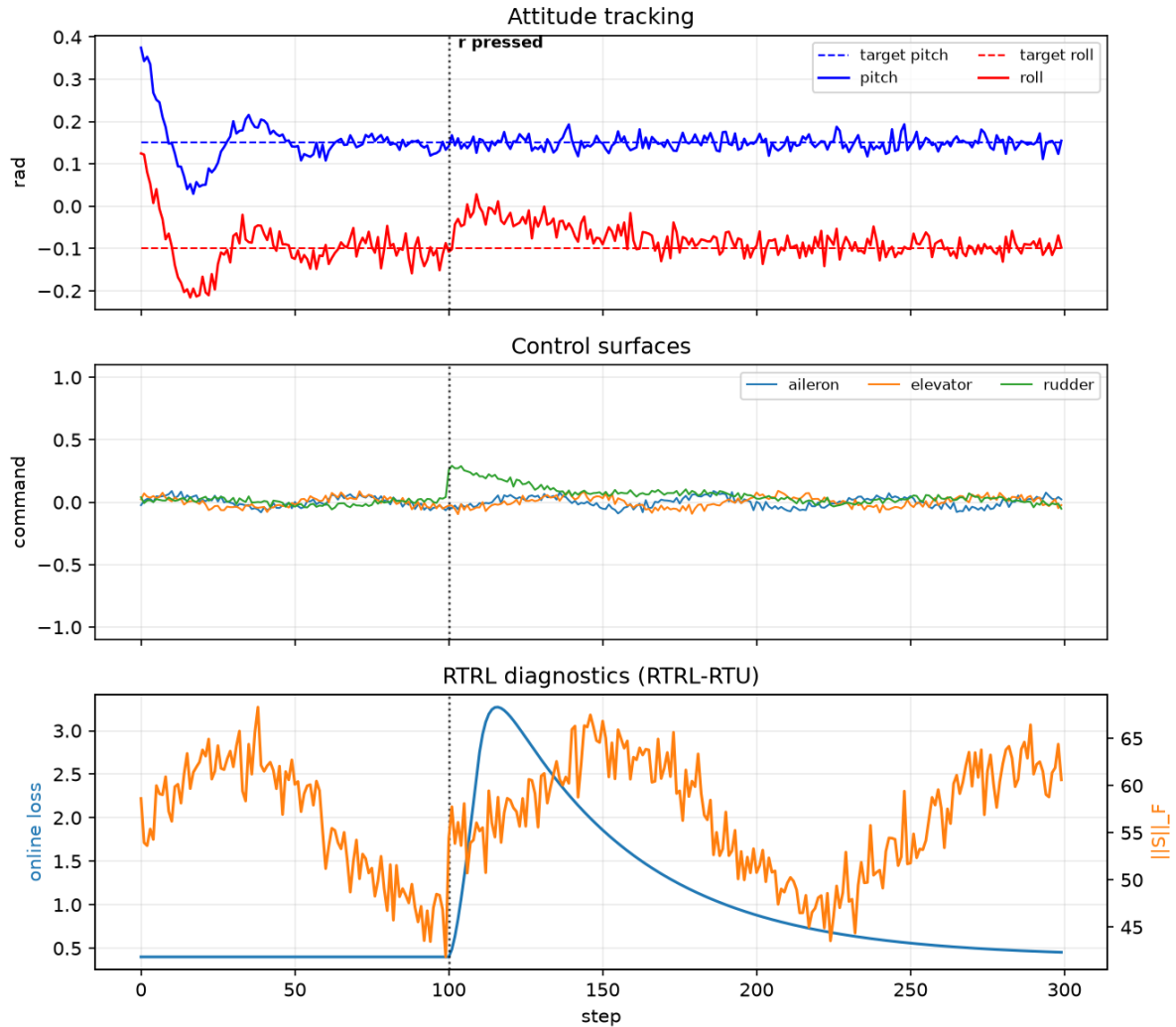


Same mechanism, on the pitch axis instead of roll: elevator authority is degraded at step 100, pitch tracking degrades and then recovers as RTRL adapts, and the elevator command shows the same compensate-then-settle pattern.

3. Fault: rudder — expected (subtler) recovery

Live demo: 'Fault: rudder' triggered mid-flight (RTRL-RTU) — expected (subtler) recovery

ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN

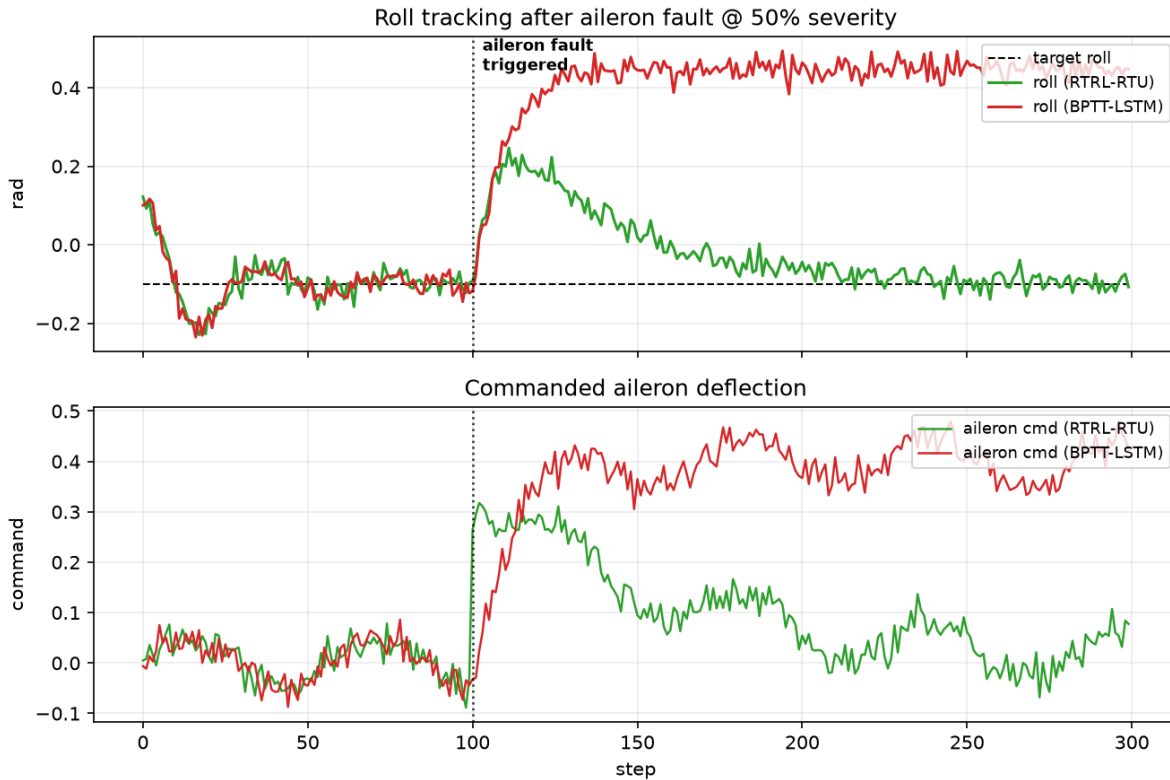


Rudder mainly affects yaw/coordination rather than the plotted pitch/roll targets directly, so its effect here is intentionally subtler than aileron/elevator — a smaller, faster-decaying bump in roll and a visible but modest change in the rudder trace itself. If you're demoing this trigger, watch the rudder line in Control surfaces and the status text's Fault line to confirm it fired.

4. The headline comparison — RTRL recovers, frozen BPTT-LSTM does not

The headline result (expected): RTRL-RTU recovers, frozen BPTT-LSTM does not

ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN

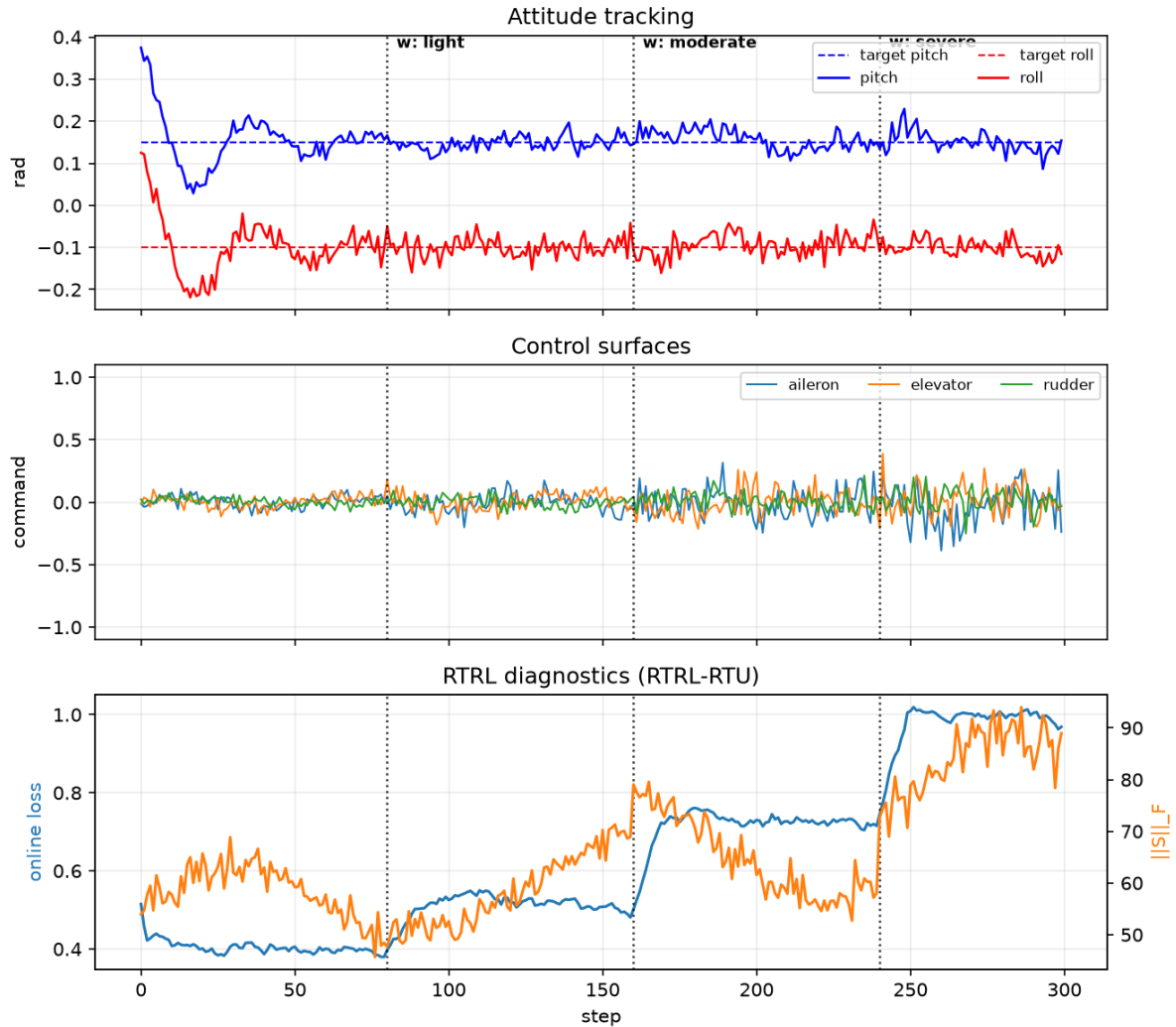


This is the project's central success criterion (per CLAUDE.md): the same aileron fault, same onset step, run against both controllers. RTRL-RTU's online weight update lets roll converge back toward the target; BPTT-LSTM's weights are frozen at deploy time, so its roll error keeps climbing and never comes back down. The aileron command tells the same story — RTRL's command eases off as it adapts, BPTT-LSTM's stays elevated because it's still fighting the same (wrong) internal model of the aircraft's dynamics.

5. Cycle wind — expected turbulence response

Live demo: 'Cycle wind' pressed 3 times (off -> light -> moderate -> severe) — expected

ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN

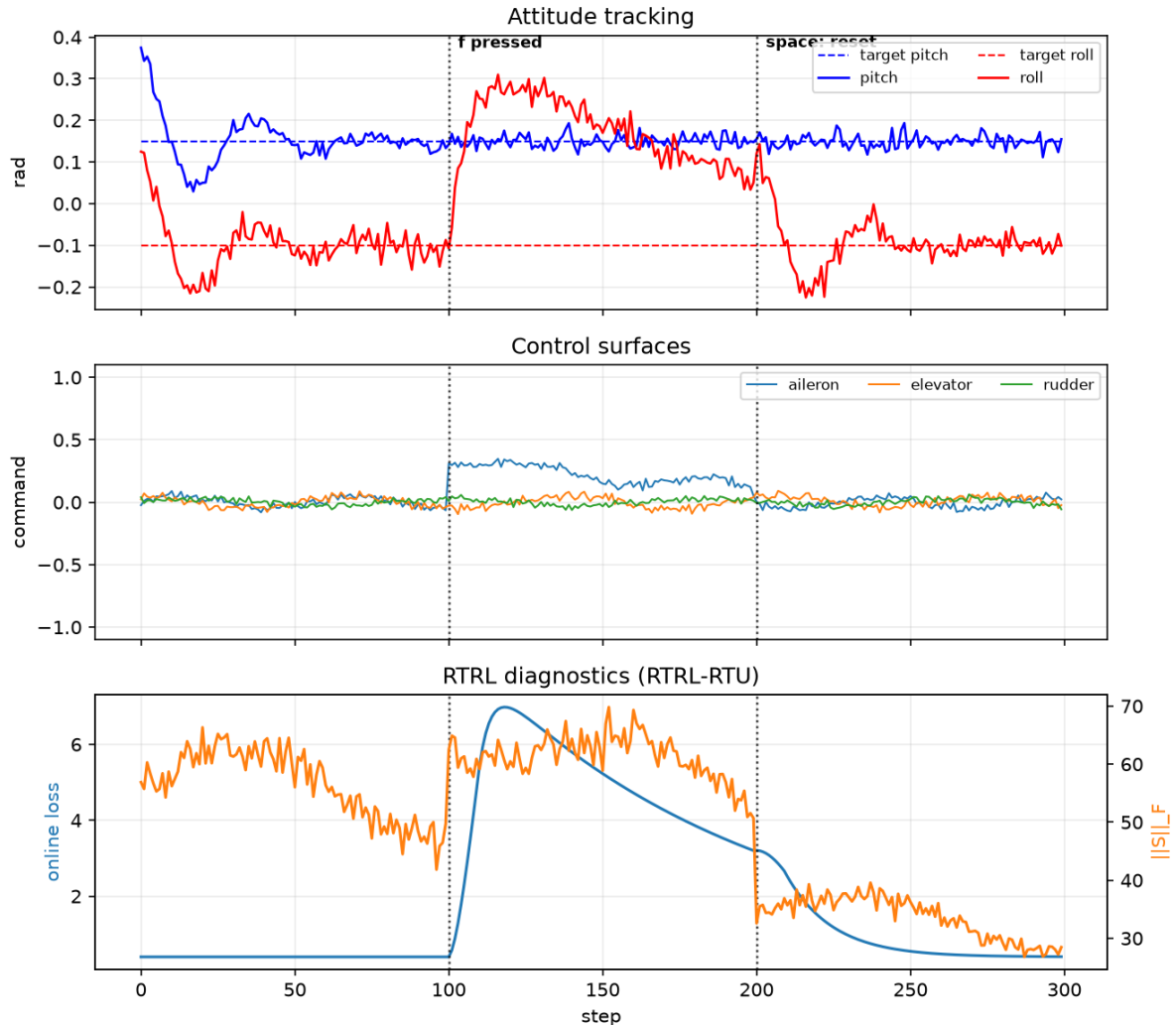


Wind severity is advanced three times (light, moderate, severe). Unlike a fault, this is ambient disturbance, not an actuator failure: pitch/roll stay centered on target throughout, but visibly noisier at each step up in severity, with more corrective jitter in the Control surfaces panel and a mild rise in online loss / sensitivity norm rather than a fault's sharp spike-then-decay.

6. Reset episode — expected discontinuity

Live demo: 'Reset episode' pressed while aileron fault was still active — expected

ILLUSTRATIVE — EXPECTED BEHAVIOR, NOT A CAPTURED RUN



An aileron fault is triggered at step 100, then "Reset episode" is pressed at step 200 while recovery was still in progress. At the reset marker: fault authority snaps back to 100% (the aileron command drops back to its clean baseline amplitude), and the RTRL diagnostics restart near their pre-fault baseline — matching the Controller ABC's contract that `reset()` clears only episode-local state (hidden state, sensitivity tensors), not the controller's learned weights. The new episode then settles from its own fresh initial condition.